

SC05 | University Timetable Optimization Engine

Difficulty: Advanced capstone **Core techniques:** Genetic Algorithm **Team:** 3-4 students

1. Project overview

Create a working decision-support product that converts **courses, rooms, faculty, cohorts and constraints** into **conflict-free timetable and quality score**. This is not a generic machine-learning exercise: the product must model uncertainty, trade-offs, and imperfect real-world conditions through **Genetic Algorithm**. The final application must be runnable by a non-technical user, explain its result visually, and demonstrate a measurable improvement over a rigid rule-only approach.

2. Constraints and non-negotiable requirements

Team 3-4 students	Experimental data 10,000+ cited records or documented scenarios	Product Runnable UI with input, processing, output and visual evidence
Repository Continuous, attributable Git commits	Deployment/UI Publicly accessible hosted product link is mandatory for M2	Submission Report, 5-minute video, presentation and individual viva

3. Why this is a Soft Computing capstone

- The problem contains uncertainty, non-linear behaviour, competing goals, or incomplete information.
- You must expose the important algorithmic elements rather than only calling a library function.
- A baseline is compulsory: random or greedy timetable. Your evidence must show what the Soft Computing solution improves.

4. Product and system architecture

Pipeline: data/scenario collection → validation and preprocessing → baseline → Genetic Algorithm model → decision/optimised output → evaluation dashboard.

Recommended stack: Python, NumPy/Pandas, Matplotlib or Plotly, Streamlit, and GitHub. Use scikit-learn only for a comparison baseline where relevant; it is not a substitute for explaining the course algorithm.

5. Data and experimental strategy

Required source: 10,000+ documented timetable scenario variations. Document dataset name, URL, licence, original size, features/targets, all preprocessing, and any scenario-generation assumptions. The minimum is 10,000 records or realistic test scenarios. Do not invent a citation: state clearly what is real and what is simulated.

6. Milestone 1 - Working Product V1 (mid-semester)

Objective: Create a locally working, evidence-backed first version of the product. M1 is a build milestone, not a documentation milestone.

Implement chromosome, fitness, selection, crossover, mutation, repair and a timetable viewer.

Functional requirements:

- Accept the stated project inputs, validate them, run the core algorithm, and return the required output.
- Display both the Soft Computing result and the baseline result in a way a user can understand.

- Save reproducible results and at least two clear visualisations.
- GA component: define chromosome encoding, population generation, fitness with penalties, selection, crossover, mutation, elitism/stopping condition, and fitness-progression plots.

M1 submission evidence:

- Repository created early, with README, requirements, source/app folders, data citation, and continuous commits.
- At least 10,000 records/scenarios documented and a reproducible preprocessing or generation step.
- Interim report with problem, input/output specification, architecture, algorithm explanation, MVP screenshots, results, limitations and next steps.
- A live M1 demo: input → processing → output. Documentation-only work is not accepted.

7. Milestone 2 - Enhanced Product V2 (end-semester)

Objective: Convert the M1 prototype into a complete, reliable and publicly hosted product with advanced experimentation.

Add multi-objective constraints and parameter studies; compare GA and heuristic quality at scale.

Functional and technical requirements:

- Complete the advanced/hybrid component and make the final application runnable from README instructions.
- Run parameter experiments rather than a single best run. Record settings, results, interpretation and selected configuration.
- Compare baseline, M1 approach, and advanced M2 approach where applicable, using hard violations, soft penalty, runtime and fitness.
- Deliberately test edge cases: noise, extreme values, contradictory inputs, unseen cases, or difficult constraints.
- Deploy the final UI on an approved public platform. Localhost-only demonstrations are not accepted.
- Publish final plots/tables, report, ~5-minute demo video, presentation material, and team contribution details.

8. Required repository evidence

- README with team, problem, architecture, dataset citation, setup/run commands, results, deployment link and limitations.
- src/ for reusable algorithms; app/ for runnable product; results/ for labelled outputs; report/ for reports; data/README.md for source information.
- Meaningful commits throughout the semester, such as “Implement fuzzy rules”, “Add GA mutation”, or “Evaluate baseline”. A final ZIP/code dump does not earn full Git marks.

9. Evaluation evidence (50 marks)

GitHub - 10 Structure, code quality, continuous commits and contribution.	Report - 10 Problem, data citation, method, implementation, experiments and results.
Video - 10 Problem, concept explanation, product demo, comparison and clarity.	Deployment & UI - 10 Public hosted link, working flow, usability, visualisation and reliability.
Viva & Presentation - 10 Technique understanding, own code, results and communication.	Total - 50 Viva marks are individual; other artefacts may be team-level.

Final reminder

This is an engineering product capstone. A strong submission makes it easy for a faculty member to run the project, inspect the evidence, compare it with the baseline, and ask each student about the part they built.